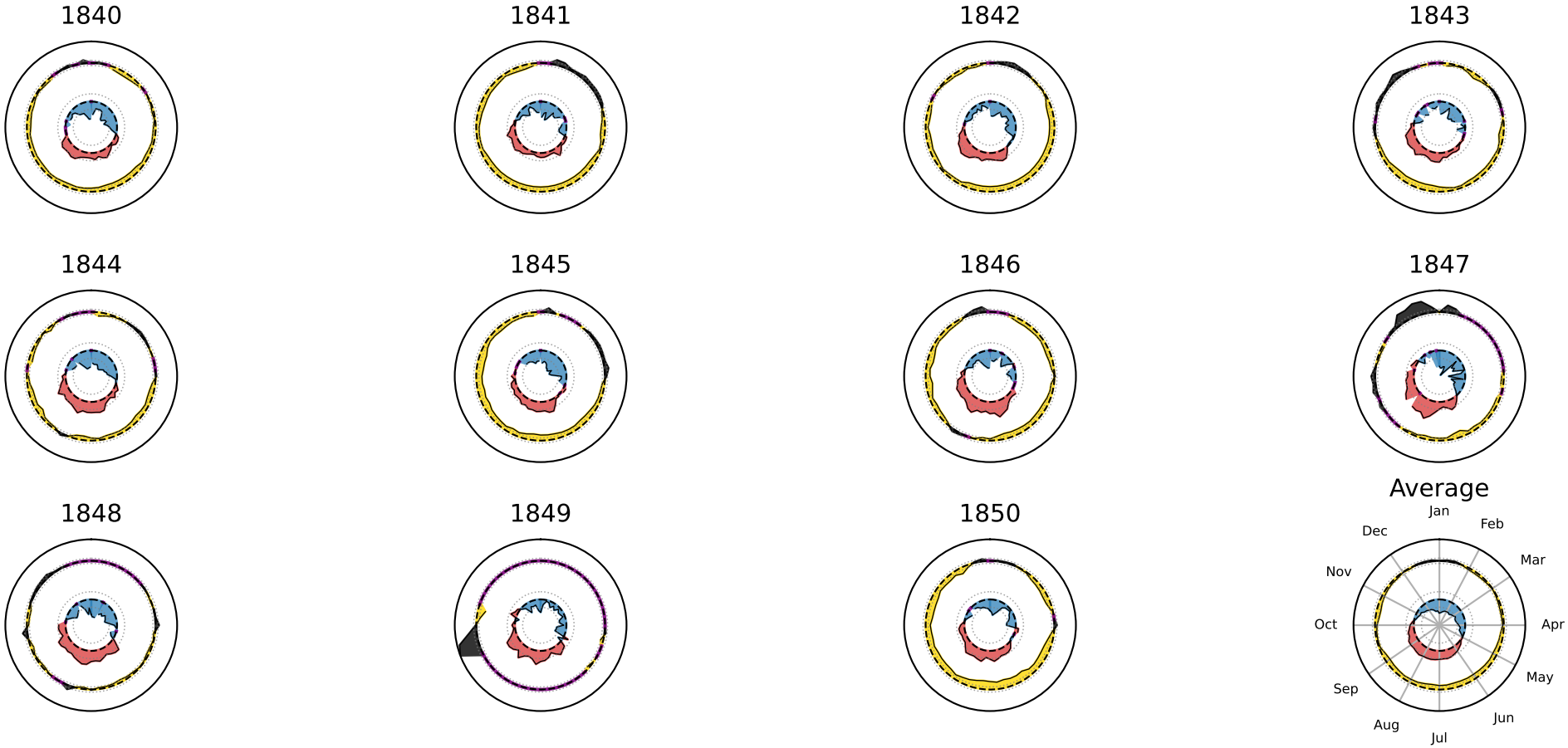
Temperature & Mortality of London (1840–1850)



Inner ring: temperature (°F) | Outer ring: weekly death counts | Dashed circles: reference values
Dotted circles show ±10°F and ±100 deaths from the references. Weeks progress clockwise from January at 0°. Purple indicates missing values filled with reference values.

Above-average temperature
Below-average temperature
Above-average mortality
Below-average mortality
Missing data

Source: Adapted from William Farr, Temperature and Mortality of London (1840–1850), published 1852.

CS7DS4 / CSU44065 Data Visualization 2026-27
Assignment 1
Student Name: Parth Verma
Student No: 26356239
Declaration: I have read and I understand the plagiarism provisions in the General Regulations of the University Calendar for the current year, found at http://www.tcd.ie/calendar. I have also completed the Online Tutorial on avoiding plagiarism 'Ready Steady Write', located at http://tcd-ie.libguides.com/plagiarism/ready-Steady-write.

A1.1 Write-up

For this visualization, Python, Pandas, NumPy, and Matplotlib were used. Python was used as the main programming language to create the actual visualization. Pandas was used to load, clean, filter, and aggregate the data from the CSV file. NumPy was used for Numerical calculations, arrays, angles, and scaling values. Finally, Matplotlib was used to create the charts, legend, labels, and general layout.

In the data preparation step, the CSV file was first read. There were missing values, so these were identified and then replaced with the reference values. They were also noted to be marked in purple at this point to highlight that the actual data is missing at those points.

The implementation of the charts was based around using polar coordinates, two concentric radial area charts, and a reusable function for each year. This allowed for efficient duplication of the charts by year, and for readable and easily comparable data.

The summary chart differs slightly from the others. Not only does it contain the weekly averages over the course of 1840 to 1850, but it also has a few notable informative features. The summary chart also contains consistent radial scales, reference circles, dotted scale guides, month labels, and the shared legend. This helps to understand the other charts while minimizing clutter on the entire visualization.

The design was chosen to mimic Farr's original decision, with a simplified presentation. This maintains the same radial layout, but reduces the clutter of additional axes and text by having the summary chart host the average data, and all additional information to be generalized to all other charts.

A1.2 - Part A

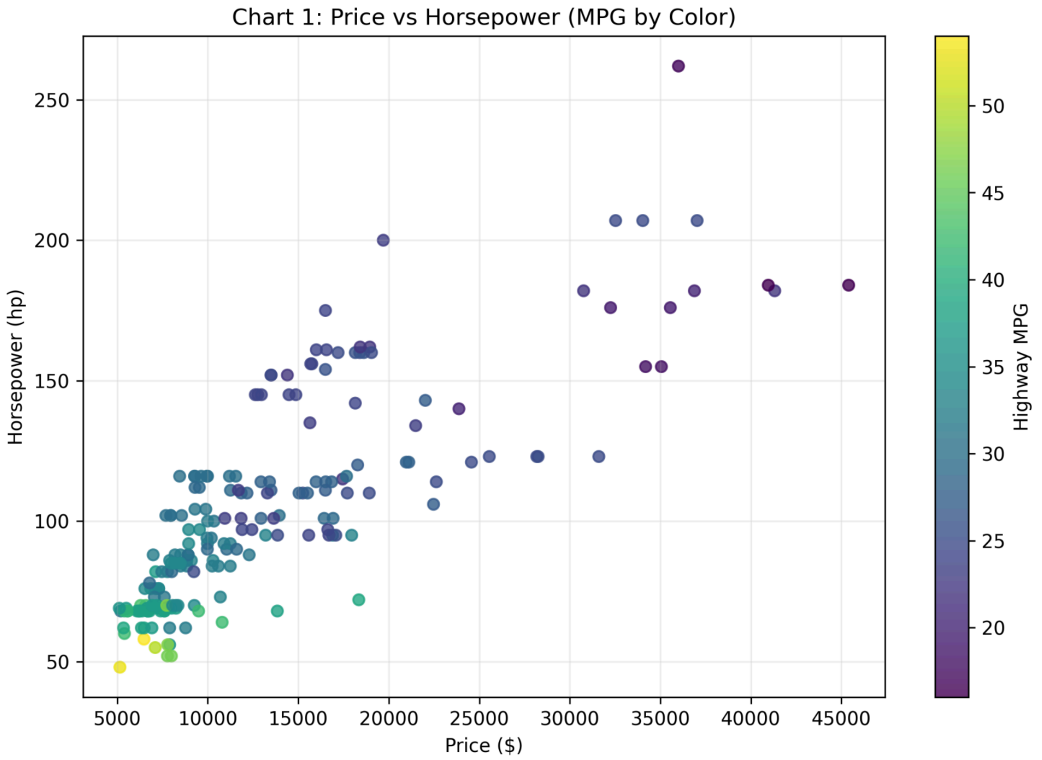


Figure 1: MPG by Color. Here, price is encoded by horizontal position and horsepower by vertical position. Highway MPG is shown by a continuous color gradient.

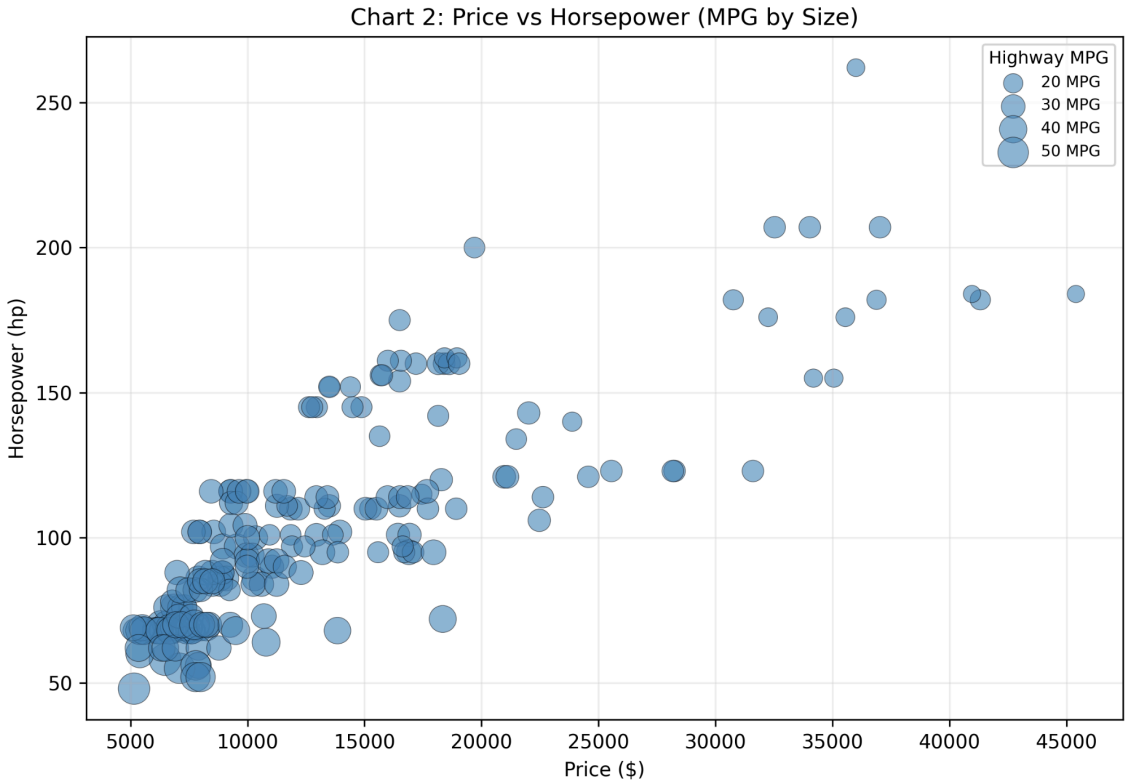


Figure 2: MPG by Size. Price and horsepower retain their positional encodings from Chart 1, while highway MPG is encoded through marker size/area.

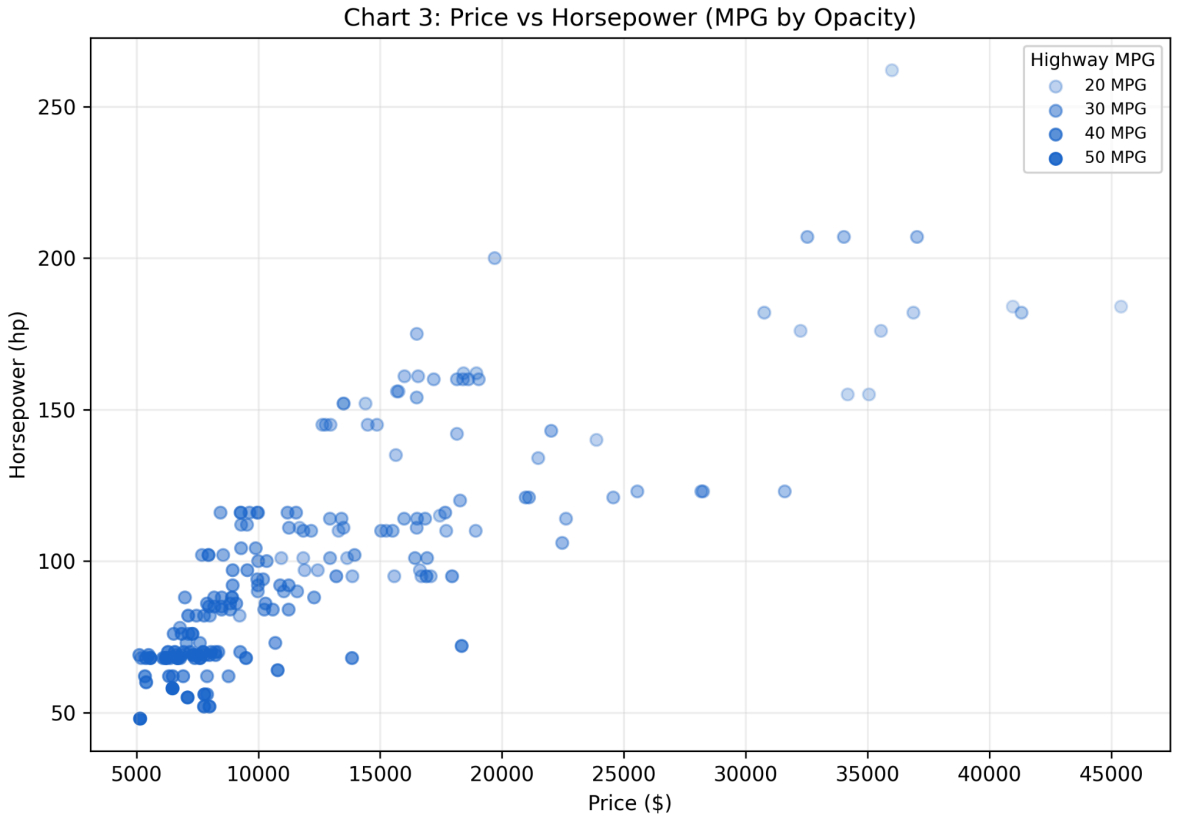


Figure 3: MPG by Opacity. Price and horsepower keep the same positional encodings as above, while highway MPG is shown through marker transparency.

In A1.2 Part A, the same tools as A1.1 were used. Python was used as the main programming language to create the visualizations. Pandas was used for loading and filtering the data. NumPy was used for mathematical computations, and Matplotlib was used in creating the charts, legends, and labels.

Charts 1, 2, and 3 represent Price (X-axis) against Horsepower (Y-axis). The difference between them is that they change the visual encoding of highway MPG. Chart 1 uses color. This is the most effective of these three charts, as the differences in fuel efficiency were much easier to visually identify. This chart had minimal data obstruction in viewing. Chart 2 is based on marker size. While this method makes the MPG differences visible, it also introduces a decent amount of overlap. This does make distinguishing individual vehicles much more difficult. Finally, Chart 3 uses opacity to show the various data points. A higher MPG is represented by a darker marker. This is not a very productive method for differentiating different points, as overlap and crowding can make judging transparency very difficult.

Overall, Chart 1 had the clearest encoding (using color) when comparing highway MPG, followed by size, and finally opacity.

A1.2 Part B

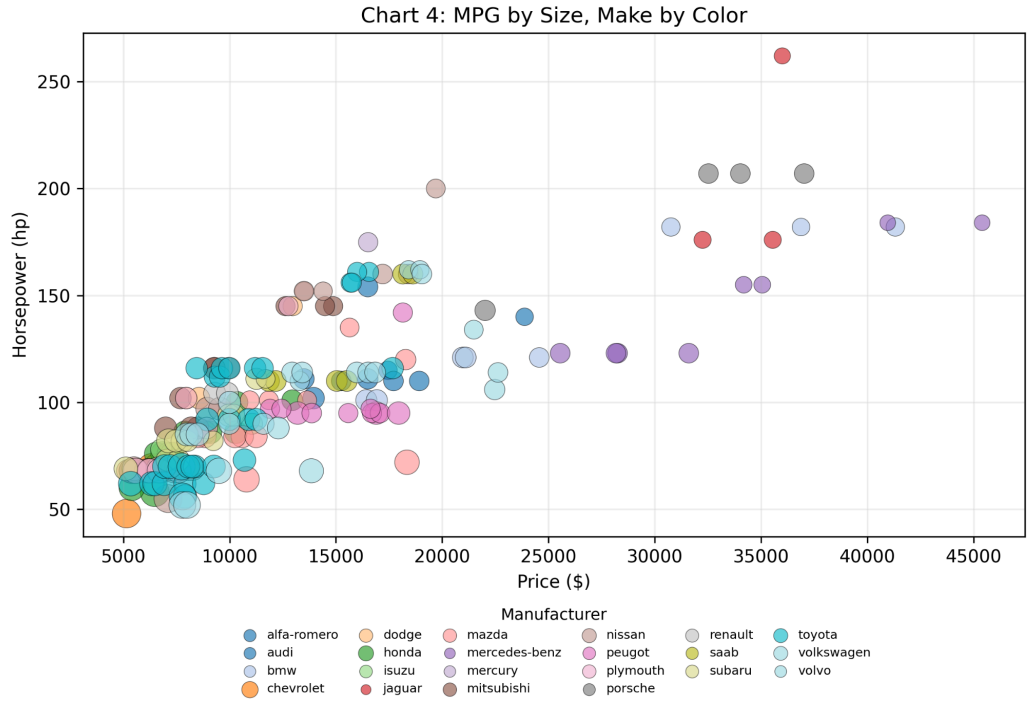


Figure 4: Make by Color. Price is encoded by horizontal position, horsepower by vertical position, highway MPG by marker area, and manufacturer by color.

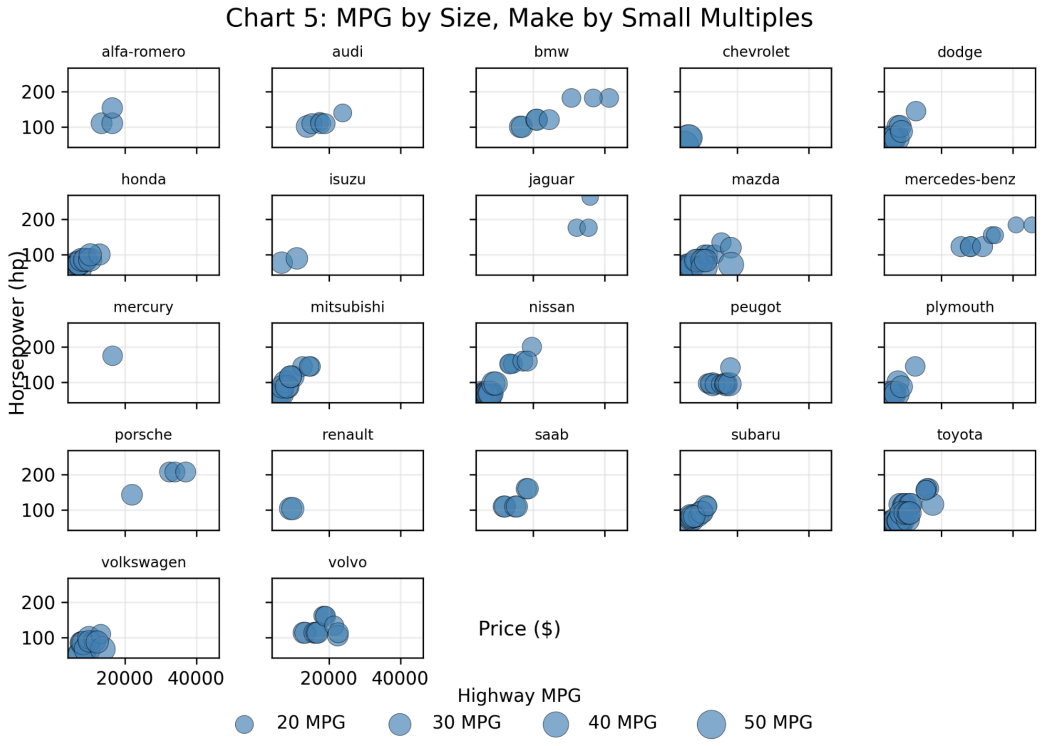


Figure 5: Make by Small Multiples. Price, horsepower, and highway MPG retain the same encodings, while manufacturers are separated into individual subplots.

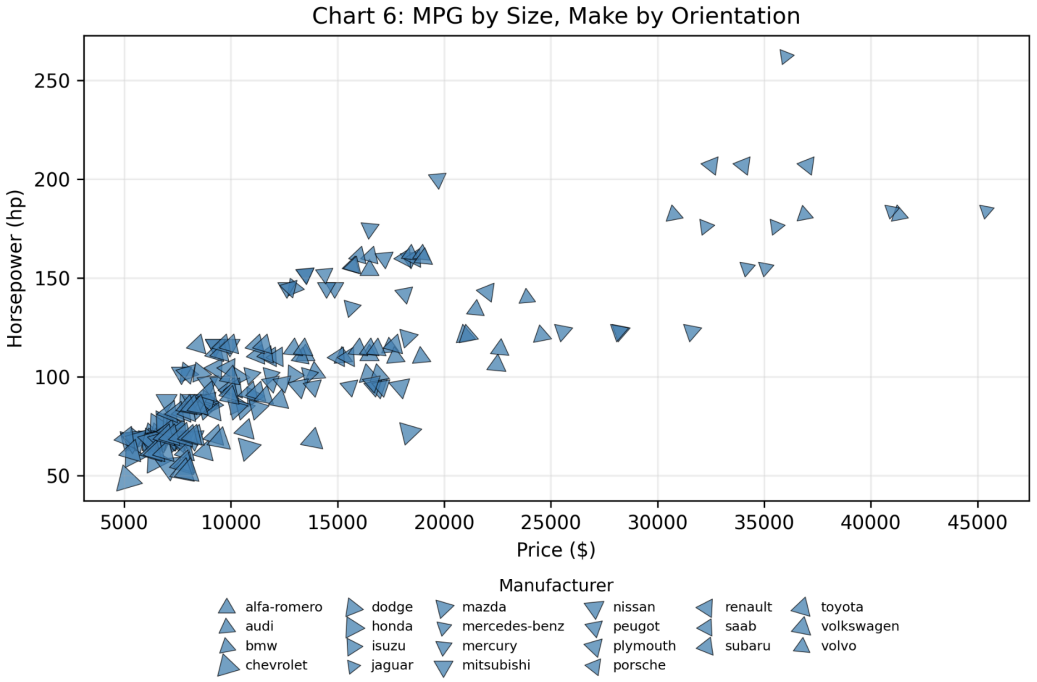


Figure 6: Make by Orientation. Price, horsepower, and highway MPG retain the same encodings, while manufacturer is represented by the rotation of triangular markers.

In A1.2 Part B, the same tools were used as in previous sections. Python was used as the main programming language to create the visualizations. Pandas was used for loading and filtering the data. NumPy was used for mathematical computations, and Matplotlib was used to create the charts, legends, and labels.

Charts 4, 5, and 6 use three methods to distinguish car manufacturers. Price, horsepower, and highway MPG are represented the same way, by horizontal position, vertical position, and marker size, respectively.

Chart 4 uses colors to identify manufacturers. This allows all vehicles to be compared in one scatter plot. However, with 22 different options, distinguishing can become difficult with similar shades. Chart 5 has an individual plot per manufacturer. This reduces overlap and allows for easier viewing of price and horsepower distributions. However, the separate panels are smaller, and thus make cross-manufacturer comparisons harder. Chart 6 uses orientation, with rotation triangles, to represent manufacturers. While this is a new visual channel, it is easily the most difficult to distinguish.

Thus, multiple panels were most effective in examining each manufacturer, and color was better for comparing them against each other.

A1.2 Part C

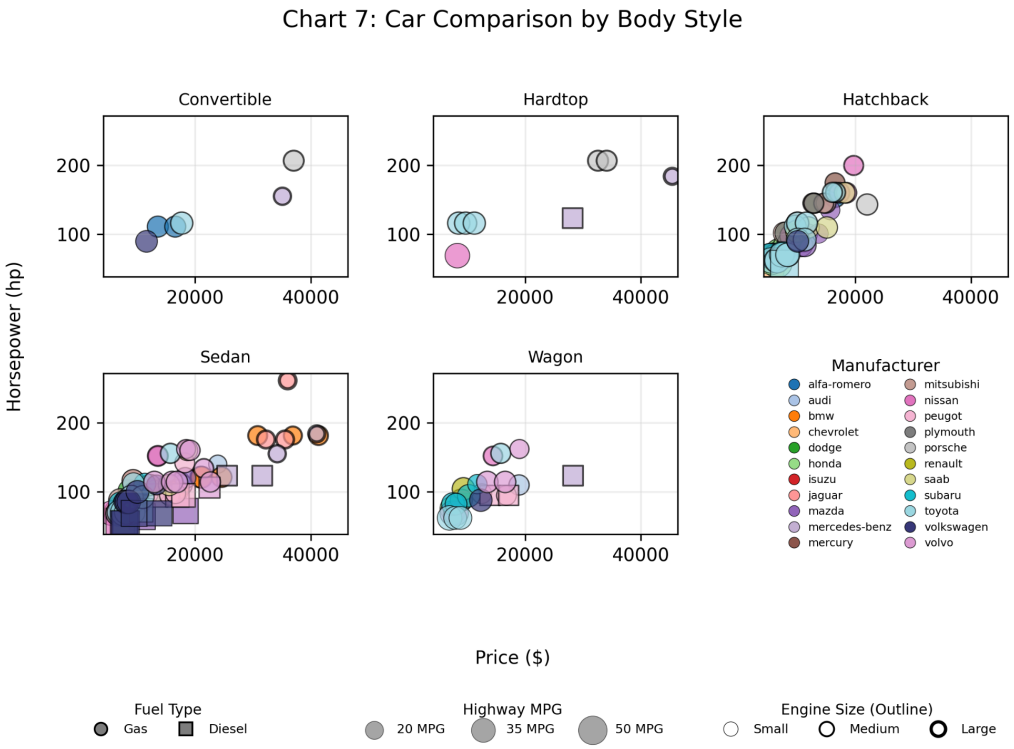


Figure 7: Multi-Attribute Car Comparison. Price is represented by horizontal position, horsepower by vertical position, highway MPG by marker area, manufacturer by color, fuel type by marker shape, engine size by outline thickness, and body style by separate panels.

In A1.2 Part C, the same tools were used as in all other sections. Python was used as the main programming language. Pandas was used for loading and filtering the data. NumPy was used for mathematical computations, and Matplotlib was used to create the charts, legends, and labels.

Chart 7 separates cars into body-style categories to support car-buying decisions. This allows potential buyers to take note of the vehicle type they would like before comparing brands on price, horsepower, and fuel efficiency. This is a good first chart to look at in the purchasing process, to get a good start on what kind of vehicle the buyer would like. This was a very useful approach, as it provides a good overview of several characteristics at once. There is also a reduced number of points per subplot, which allows for easier individual comparisons.

It is important to note that having these various attributes does increase visual complexity. The colors corresponding to some manufacturers may be difficult to tell apart (with a number as large as 22). Additionally, engine-size differences shown with outline thickness can also be difficult to identify. Thus, while the visualization does contain a myriad of information, it is more complex to read when compared to graphs in Parts A and B.